

Типы индексов

Создание индекса в словаре данных иницирует для PostgreSQL соответствующую команду [CREATE INDEX](#), параметр которой **method** (название метода индексирования) — определяет алгоритм чтения индекса:

```
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ [ IF NOT EXIST ] name ] ON table_name [ USING method ]  
({ column_name | ( expression ) } [ COLLATE collation ] [ opclass ] [ ASC | DESC ]  
[ NULLS { FIRST | LAST } ] [ , ... ] )  
[ WITH ( storage_parameter = value [ , ... ] ) ]  
[ TABLESPACE tablespace_name ]  
[ WHERE predicate ]
```

Каждый тип индекса определяется алгоритмом, ориентированным на определенный тип запроса. Для оптимизации запросов используется четыре типа индексов, поддерживаемых PostgreSQL: B-tree, HASH, GiST и GIN.

В Genie тип индекса устанавливается с помощью опции *Method* (метод индексирования):

The screenshot shows the Dbindex application window. The 'Name' field is 'Manufacturer'. Under the 'Fields' section, there is a table with columns 'Kind' and 'Name', containing one row with 'Field' and 'Manufacturer'. The 'Predicate' field is empty. The 'Is unique' checkbox is unchecked. The 'Include null values' checkbox is checked. The 'Method' field is highlighted with a red box and contains the value 'btree'. The 'Index parameters' and 'Comment' fields are empty.

B-tree

B-tree — создается по умолчанию, наиболее часто используемый тип индексов, эффективен в большинстве случаев.

Организован как [сбалансированное дерево](#) упорядоченных ключей.

B-tree работают в условиях на равенство и в проверках диапазонов с данными, которые можно отсортировать в некотором порядке, они позволяют использовать следующие операторы:

< <= = >= >

Поиск по индексу B-tree также выполняется при обработке конструкций, представимых как сочетание этих операторов, например BETWEEN и IN. Кроме того, индексы B-tree могут использоваться и в условиях IS NULL и IS NOT NULL по индексированным полям.

Оптимизатор может использовать эти индексы в запросах с [операторами поиска по шаблону](#):

- LIKE и ~ — если шаблон определяется константой и привязан к началу строки:

```
col LIKE 'foo%'
```

или

```
col ~ '^foo'
```

но не так:

```
col LIKE '%bar'
```

- `ILIKE` и `~*` — если шаблон начинается с символов, не подверженных преобразованию регистра.

B-tree могут также применяться для получения данных, отсортированных по порядку. Это не всегда быстрее простого сканирования и сортировки, но иногда бывает полезно.

HASH

Hash-индексы предполагают хранение не самих значений, а их хэшей. Таким образом, при запросах с использованием Hash-индексов сравниваться будут не искомое со значениями поля, а хэш от искомого значения с хэшами полей.

Они могут подойти для индексов из больших полей:

- уменьшается размер,
- увеличивается скорость обработки.

Однако, нужно учитывать, что из-за нелинейности хэш-функций данный индекс нельзя сортировать по значению, и, следовательно, невозможно использовать в операциях сравнения больше/меньше и "is null"; индекс работает только с простыми условиями равенства. Кроме того, так как хэши не уникальны, то для совпадающих хэшей применяются [методы разрешения коллизий](#).



С использованием Hash-индексов следует учитывать два важных момента:

- операции с Hash-индексами в настоящее время не проходят через [Журнал опережающей записи \(WAL\)](#), так что после аварийной остановки базы данных может потребоваться их переиндексация (команда [REINDEX](#));
- не рекомендуется использовать Hash-индексы в условиях применения [поточковой или файловой репликации](#): после начальной копии изменения в Hash-индексах не переносятся, так что в последующих запросах они будут давать неправильные ответы.

Таким образом, Hash-индексы **НЕ РЕКОМЕНДОВАНЫ** к использованию.

Планируя применение таких индексов, разработчик должен четко понимать, что выгода от их использования гарантированно перекроет возможные проблемы.

GiST и GIN

Эти два типа индексов используются для типов данных ["tsvector"](#) и ["tsquery"](#), которые разработаны для поддержки [полнотекстового поиска](#).

GiST

[GiST-индексы \(Generalized Search Tree, Обобщённое Дерево Поиска\)](#) представляют собой структуры данных, у которых для каждого вектора, описывающего все лексемы документа, создается [битовая сигнатура](#), описывающая, какие лексемы содержатся в данном документе.

GiST-индексы могут использоваться для ускорения поиска текстовых данных, для ускорения поиска в определенном временном интервале а в общем случае применяются для всех видов нерегулярных структур данных (целочисленные массивы, спектральные данные и т.д.), к которым неприменимо обычное индексирование B-Tree. Используется для типов данных ["tsvector"](#) или ["tsquery"](#). Стандартный дистрибутив PostgreSQL позволяет применять индексы в запросах с [операторами](#):

```
<<  &<  &>  >>  <<|  &<|  |&>  |>>  @>  <@  ~=  &&
```

GiST-индексы также могут оптимизировать поиск [ближайшего соседа](#) — [k-nn search](#). Возможность такого применения индекса зависит от используемого [класса оператора](#).

GIN

[GIN-индексы \(Generalized Inverted Index, Обобщённый Инвертированный Индекс\)](#) представляют собой структуры данных, у которых ключом является лексема, а значением — сортированный список идентификаторов документов, которые содержат эту лексему.

Используется для типов данных ["tsvector"](#).

Подобно GiST, индексы GIN могут использоваться для ускорения поиска текстовых данных. Стандартный дистрибутив PostgreSQL позволяет применять индексы в запросах с [операторами](#):

```
<@ @> = &&
```

Использование этого индекса в БД для индексирования изменяющихся документов затруднительно, так как любые изменения в документах (добавление, удаление или обновление) приводят к большому количеству обновлений индекса (которое зависит от количества новых уникальных лексем). В условиях массовых вставок, например, [рекомендуется](#) удалить и пересоздать индекс. Поэтому рекомендуется использовать GIN для неменяющихся (меняющихся редко) коллекций документов.

Индексы GIN и GiST не позволяют получить данные в отсортированном виде.

Индексы этих двух видов значительно различаются по быстродействию, поэтому важно понимать их особенности:

- Поиск по индексу GIN примерно в три раза быстрее, чем по GiST.
- Индексы GIN строятся примерно в три раза медленнее, чем GiST (GIN не "любит" большого количества вставок).
- Индексы GIN обновляются в десять раз медленнее, чем GiST
- Индексы GIN обычно в два-три раза больше, чем GiST

Основное правило такое: GIN индекс используется для редко изменяющихся данных (при большом количестве добавления, а GiST индекс — для часто изменяющихся данных.